

Programmation et Langages 1

Devoir surveillé du 28 octobre 2021

Durée : une heure – Tous documents autorisés**Exercice 1 (~ 5 points)**

Un entier naturel x est dit *premier* si et seulement si il n'est divisible que par 1 et par x . Un algorithme simple permettant de décider si un entier x est premier ou non consiste donc à :

1. **rechercher** un entier i dans $[2, x - 1]$ tel que x modulo i soit égal à 0.
2. si un tel entier i existe, alors x n'est pas premier.

Ecrire un programme C qui :

1. lit un entier positif x au clavier
2. affiche à l'écran si x est ou non premier

Exercice 2 (~ 5 points)

On souhaite écrire un programme qui compte le nombre d'occurrences de lettres (minuscules et non accentuées) dans un mot. Par exemple, le mot "caractere" contient 2 occurrences de la lettre 'a', 2 occurrences de la lettre 'c', 2 occurrences de la lettre 'e', 2 occurrences de la lettre 'r' et 1 occurrence de la lettre 't'.

Pour produire ce résultat on propose d'utiliser l'algorithme suivant :

lexique

N : une constante entière > 0

NbOcc : un tableau sur ['a' .. 'z'] d'entiers ≥ 0

mot : une chaîne de au plus N caractères terminée par '\0'

i : un entier ≥ 0

debut

lire (mot)

pour j parcourant 'a' .. 'z' : nbOcc_j $\leftarrow$ 0

i $\leftarrow$ 0

tantque mot_i $\neq$ '\0'

nbOcc_{mot_i} $\leftarrow$ nbOcc_{mot_i} + 1

i $\leftarrow$ i+1

pour j parcourant 'a' .. 'z' : ecrire (nbOcc_j)

fin

On rappelle qu'en C un tableau de taille N est indicé par des entiers sur $[0, N-1]$. Pour implémenter le lexique précédent on doit donc représenter les indices sur $['a' \dots 'z']$ du tableau `nbOcc` par des indices sur $[0, 25]$ (en supposant un alphabet de 26 lettres minuscules et non accentuées). L'expression algorithmique `nbOccj` (où j est une lettre minuscule non accentuée) pourra alors se traduire en langage C par `nbOcc[j - 'a']`.

Donner une traduction complète en C du lexique et de l'algorithme ci-dessus.

Exercice 3 (~ 10 points)

On souhaite écrire un programme C permettant de représenter et gérer un **répertoire téléphonique**. On considère dans la suite qu'un numéro de téléphone comporte toujours entre 2 et 10 chiffres, et qu'il débute toujours par le chiffre 0 lorsqu'il comporte 10 chiffres.

Q1. On propose trois solutions pour coder en C un numéro de téléphone :

1. par un tableau de $N=10$ entiers, avec longueur explicite
2. par une chaîne de caractères (terminée par `'\0'`)
3. par un entier non signé sur 32 bits

Ecrivez la définition en C du type `Tnumtel` correspondant à chacune de ces trois solutions. Vous justifierez pourquoi la solution 3 est envisageable.

Q2. Pour implémenter un répertoire téléphonique comportant au plus N contacts on propose d'utiliser une séquence de "contacts" (avec longueur explicite) selon le lexique suivant :

lexique

`N` : une constante entière > 0
`TNom` : chaîne de au plus 20 caractères, terminée par `'\0'`
`TContact` : le type `< nom : TNom, tel : Tnumtel >`
`Repertoire` : un tableau sur $[0..N]$ de `Tcontact`
`Nbcontact` : un entier sur $[0..N]$ { le nombre de contacts présents dans `Repertoire` }

Ecrivez un fragment de code C qui permet de :

1. lire au clavier une chaîne `unNom` (supposée comporter moins de 20 caractères)
2. **rechercher** dans `Repertoire` s'il existe un contact de nom `unNom` et afficher son numéro de téléphone si c'est le cas

Vous pouvez choisir une des définitions proposées à la question **Q1** pour le type `Tnumtel`. Vous pouvez utiliser la fonction C `int strcmp(const char s1[], const char s2[])` de la librairie `<string.h>` qui permet de comparer deux chaînes de caractères `s1` et `s2` en renvoyant la valeur 0 si et seulement si les deux chaînes sont identiques.

Q3. (hors-barème, à traiter en dernier !) Ecrivez un fragment de code C qui permet de :

- lire au clavier un nouveau "contact" (nom et numéro de téléphone);
- l'ajouter dans `Repertoire` (on supposera qu'il n'est pas déjà présent)